

MOBILE APP SUCCESS PREDICTION

GROUP-03:

AVS

JAAJS

JS

KGP

MA

Table of Contents

Table of Contents	0
Introduction:	2
Dataset:	2
Methods	3
Data Preparation	4
Data Transformation	4
Normalizing	4
Data Manipulation	5
Class Balancing	5
Data Splitting	6
Data Exploration:	6
Machine Learning Models	12
Binary classification	12
Multiclass Classification	13
1. Logistic regression:	13
2. Decision tree:	15
3. Random Forest:	17
4. Support vector machine:	19
6. Naive bayes:	22
ROC CURVE	23
Evaluation method:	25
Accuracy:	26
Precision:	26
Recall:	26
F1_Score:	27
Conclusion:	29
Appendix	30
References:	30
Author Contribution:	31
Script details:	32

Introduction:

The constantly evolving mobile landscape is challenging to navigate. The proportion of mobile over desktop is escalating. Android occupies around 53.2% of the smartphone market, while iOS covers 43%. To encourage more people to download the app, the developer must ensure that users can quickly find it. Mobile app analytics is an excellent instrument to understand the current strategy for driving future user growth and retention. With several million apps available nowadays, this dataset aids in gaining insights and information about the top trending apps in the iOS app store. As a result, developers and stakeholders can obtain answers to significant questions such as:

1. How efficiently insights from the Apple App Store dataset can be utilized to inform market analysis, guide app development strategies, and enhance overall user experience?
2. How developers can use it in tailoring their applications to meet the specific needs and expectations of users?
3. What specific information is crucial for marketers, app developers, and other stakeholders to foster the development of profitable and user-friendly applications?

Dataset:

The dataset used is available on Kaggle and consists of information on over 7000 Apple iOS mobile applications. The information was obtained from the Apple Inc website's iTunes Search API. Period of collecting data (via API): July 2017 and the data set contains 7197 rows and 16 columns. Data about approximately 7,000 iOS apps from the App Store was collected in July 2017 and is available via this **Mobile App Store (7200 apps)**

The table below indicates the metadata for the dataset:

Column Name	Description
id	App ID
track_name	App Name
size_bytes	Size in bytes
currency	Currency Type
price	Price amount
ratingcounttot	User Rating counts (for all versions)
ratingcountver	User Rating counts (for current version)
user_rating	Average User Rating value (for all versions)
userratingver	Average User Rating value (for current version)
ver	Latest version code
cont_rating	Content Rating
prime_genre	Primary Genre
sup_devices.num	Number of supporting devices
ipadSc_urls.num	Number of screenshots showed for display
lang.num	Number of supported languages
vpp_lic	Vpp Device Based Licensing Enabled
size_bytes	Memory size (in bytes)
app_desc	Application description

Table 1: Meta data from Dataset

Problem Definition

App developers must maximize their apps' visibility and comprehend user engagement in the ever-changing world of mobile applications, where mobile usage is increasingly more common than desktop. With iOS coming in at 43% and Android at 53.2% respectively, the two operating systems still dominate the smartphone market, but the iOS App Store is still a major distribution channel for apps. This dataset, which was obtained from the iTunes Search API in July 2017, includes information on more than 7,000 Apple iOS mobile applications. The dimensions of the dataset include app ID, name, size, pricing, version information, user ratings, content rating, primary genre, and several other attributes. Using machine learning and analytics techniques to identify patterns in this dataset that influence download and user rating metrics is the main challenge.

The objective is to create a reliable system that can offer insights into the variables affecting user engagement, app growth, and future user retention for iOS apps. The goal is to better understand what makes an app successful in terms of user reception and to find patterns that set apart the top trending apps in the iOS App Store by investigating the relationships between app statistics and user ratings. The problem also involves the comparison of app statistics across different groups, aiming to uncover trends and preferences that can inform developers' strategies.

Methods

The key phases include Business Understanding, Data Understanding, Data Preparation, Modelling, Evaluation, and Deployment. Using this knowledge, we can forecast what factors contribute to an app's performance in terms of user rating and other categories.

Below is a breakdown of the Business Analytics tasks in series,

- **Environment Setup:**

Import necessary libraries and load the dataset for analysis.

- **Data Cleaning:**

Address missing values, outliers, and duplicates in the dataset to ensure data integrity.

- **Feature engineering:**

Creating new features or modifying existing ones for the requirement of business.

- **Exploratory Data Analysis (EDA):**

Explore the dataset to understand its variables, distribution, and relationships through visualization and statistical summaries.

- **Data Splitting and Cross-Validation:**

Divide the dataset into training and testing sets using the train-test split method and employ cross-validation to ensure robust model performance.

- **Model Implementation:**

Implement various machine learning models suitable for the analysis, considering factors like decision trees, random forests, logistic regression, Support Vector Machine, Naïve Bayes and gradient boosting.

- **Hyperparameter Tuning:**

Fine-tune the parameters of the model through hyperparameter tuning to optimize its performance.

- **Model Evaluation:**

Evaluate the performance of the model using relevant metrics, such as accuracy, precision, recall, and F1 score, to assess its effectiveness in addressing the objectives of the analysis.

Data Preparation

Data preparation constitutes the essential steps of handling, manipulating, and refining a dataset to optimize it for analysis. In the context of this project, the entire data preparation is conducted as a localized solution. This involves a workflow to address tasks such as data cleaning, handling missing values, handling outliers, formatting variables, and transforming the dataset to ensure its coherence and readiness for subsequent analysis. The essential steps involved in preparing the data for modeling using the Apple App Store dataset. Removing Insignificant Variables, Splitting Data into Dependent and Independent Variables, Encoding Categorical Values, Scaling Numerical Values, Merging Scaled Numerical Columns and Encoded Categorical Columns, Combining Features and Target to Create the Final Dataframe, Feature Engineering, Summary of Class Distribution. The process involves in the creation of two distinct dataframes—one encompassing both categorical and numerical values for models like Naive Bayes, Decision Trees, Random Forests, and Logistic Regression, and another tailored for Support Vector Machines and k-Nearest Neighbors, comprising only numerical columns with encoded categorical values. Feature engineering is applied to introduce additional insights, and the class distribution is summarized to understand the balance within the target variable.

Data Transformation

The data transformation steps aim to preprocess the dataset, ensuring that it is in a suitable format for the subsequent application of machine learning models. The transformations enhance the quality of the features, addressing issues related to the nature of the data and facilitating more effective modeling. The transformation processes include One-Hot Encoding, Min-Max Scaling, Combining Scaled Numerical and Encoded Categorical Columns.

Normalizing

Normalization is a technique used to rescale numeric features to a standard range, often between 0 and 1, to ensure that all features contribute equally to the analysis and prevent biases caused by variables with larger ranges. Min-Max scaling is applied to normalize the numerical values.

Data Manipulation

Data manipulation involves organizing and refining a dataset to make it suitable for analysis. In our project various data manipulation tasks are executed. Initially, irrelevant columns are removed to simplify the dataset. Subsequently, the dataset is split into features and a target variable representing app success. Categorical variables are transformed using one-hot encoding, and numerical values are scaled to ensure consistent analysis. The scaled numerical and encoded categorical columns are then merged to create a comprehensive feature set. Finally, the features and target variable are combined to form the final dataset for subsequent modeling and analysis.

Class Balancing

Class balancing is the process of fixing imbalances in the distribution of classes within a dataset, particularly in the context of classification problems. A balanced dataset has roughly the equal number of instances for each class, which helps prevent the model from being biased towards the majority class.

Imbalances can develop when one class outnumbers another, causing the model to prioritise accuracy on the dominant class while potentially failing on minority classes. Class balancing seeks to guarantee that all classes receive equal and accurate learning.

Class	Actual data
0	2415
1	4780

- Under sampling:

In machine learning, under sampling is a strategy that fixes the class imbalance by lowering the number of instances in the over-represented (majority) class. This is done to ensure that the distribution of classes within the dataset is more balanced, preventing the model from becoming slanted towards the dominant class during training. Under sample was done for Logistic regression (K fold), Random Forest, Decision tree (K fold), SVM (K fold), KNN (K fold) and Naïve bayes (K fold).

Class	Record Count
0	2415
1	2415

- Over sampling:

In machine learning, oversampling is a strategy used to address class imbalance by increasing the number of examples in the under-represented (minority) class. This contributes to a more equal distribution of classes within the dataset, which prevents the model from becoming biased towards the dominant class during training. Over sampling was done for Logistic regression (K fold), Random Forest, Decision tree (K fold), SVM (K fold), KNN (K fold) and Naïve bayes (K fold).

Class	Record Count
0	4830
1	4780

Data Splitting

One of the final steps in preparing a dataset for an ML algorithm is to divide it into test and train data. We have split the data into train and test data. We will be training the models using train data and then test them with the test data. We have taken 70% of the data for training and 30% of the data for testing. Additionally, we have used the method of K-fold cross validation for splitting the data to ensure the model's performance is tested correctly. We have implemented random forest model with 70:30 data split and we have implemented decision tree, KNN , SVM , Naïve bayes and Logistic regression using K-fold cross validation.

Data Exploration:

The initial stage of a machine learning model is data exploration, in which the raw dataset is carefully analysed and visualised to understand its features, reveal trends, and identify potential concerns such as outliers or missing numbers. This method informs the model's succeeding processes, allowing for more informed judgements concerning data preparation, feature engineering, and model development.

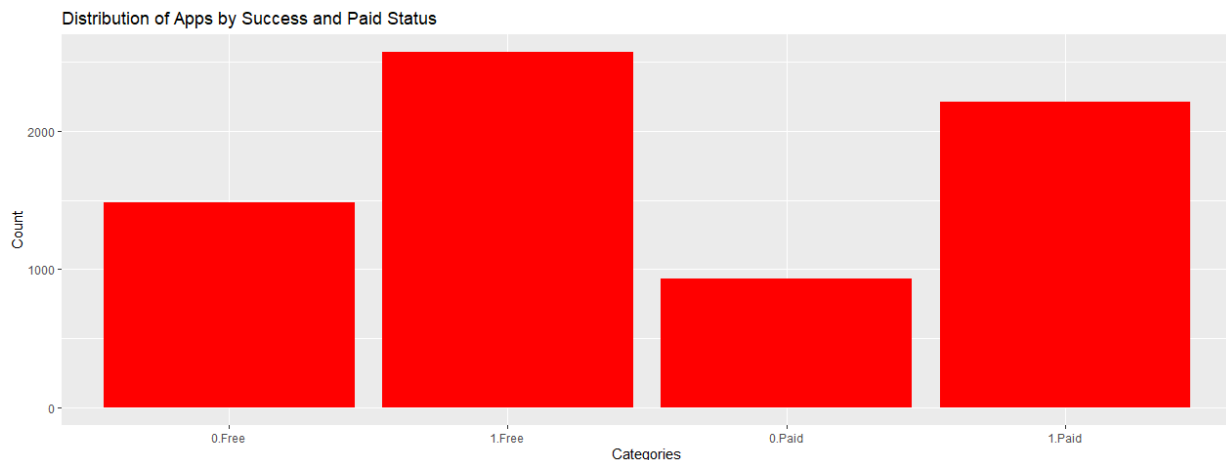


FIG 1: Distribution of apps by success and paid status

Result: Non paid apps are successful when compared.



FIG 2: User rating by Prime genre and Paid Status

Result: Catalogs, medical and shopping-paid category have lower ranges than other genres, while music, navigation, reference, and weather apps in the non-paid category also exhibit lower ranges compared to other genres.

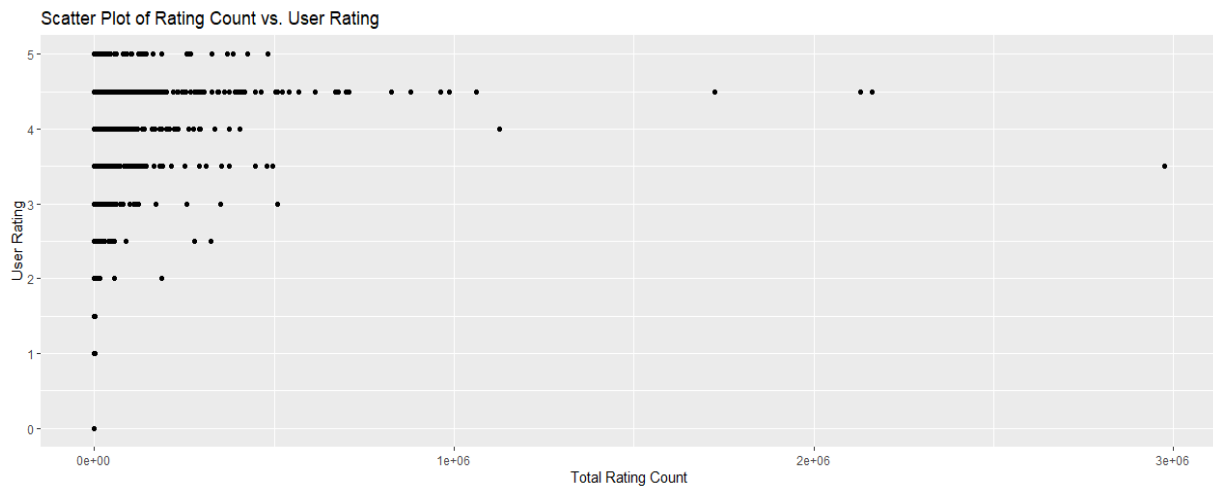


FIG 3: Comparison Between Total Rating Count Vs User Rating

Result: User Rating- 4.5 Rating Seems To Have The Maximum Total Rating Count

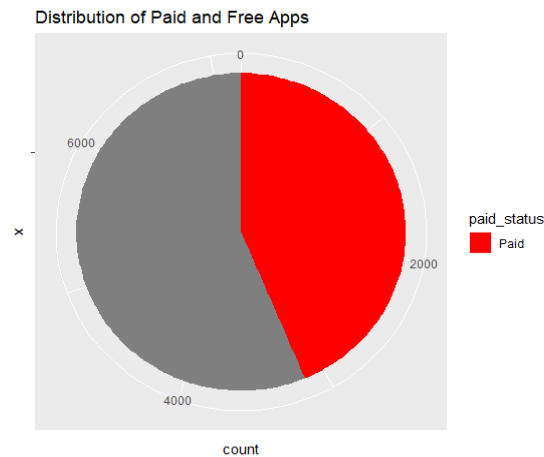


FIG 4 : Distribution of Paid and free apps

Result: There are 3,141 Paid apps and 4,054 free apps.

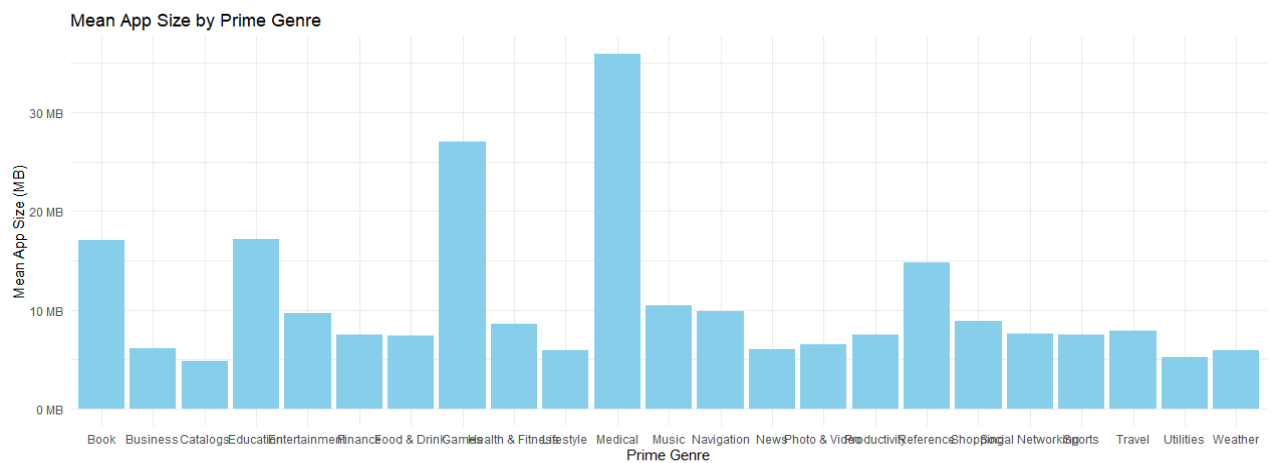


FIG 5: Mean app size by prime genre.

Result: prime genre which is under medical has the highest mean app size

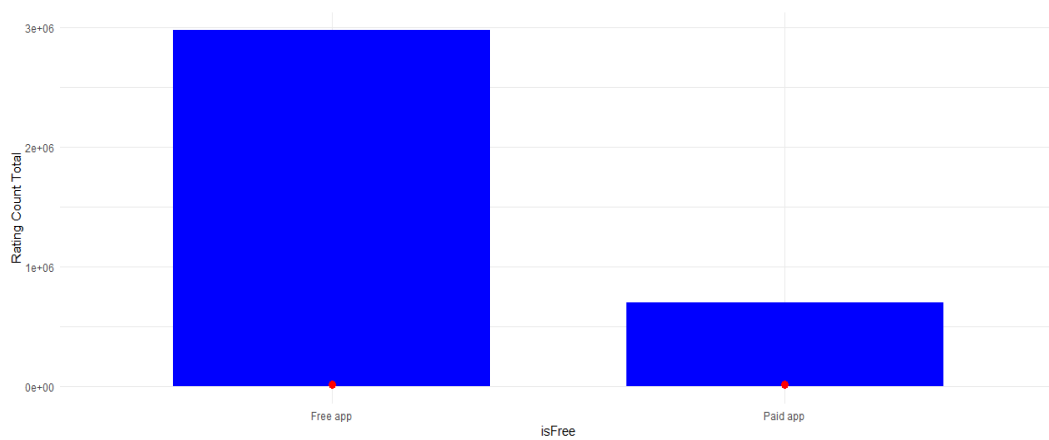


FIG 6 : Comparison between paid status and total rating count

Result: Free app seems to have the maximum total rating count

Distribution of User Ratings

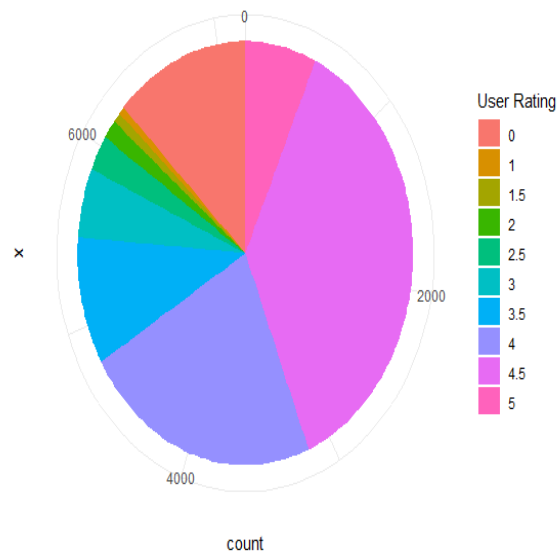


FIG 7 : Count of User Rating

Result: User rating from 4.5 to 5 seems to have the maximum overall count.

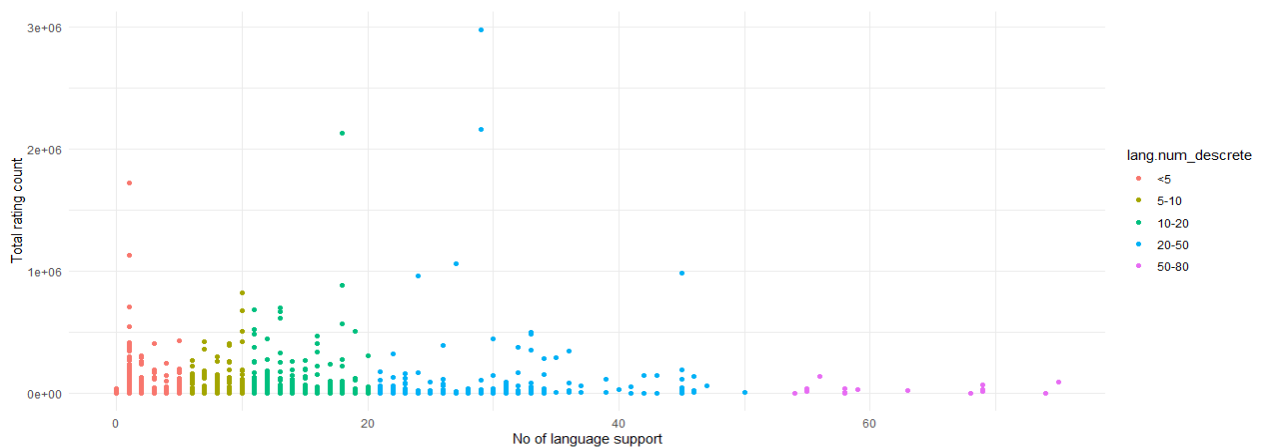


FIG 8 : Comparison between Language Supported and total rating count

Result: apps having language support around 27-29 had the highest rating count

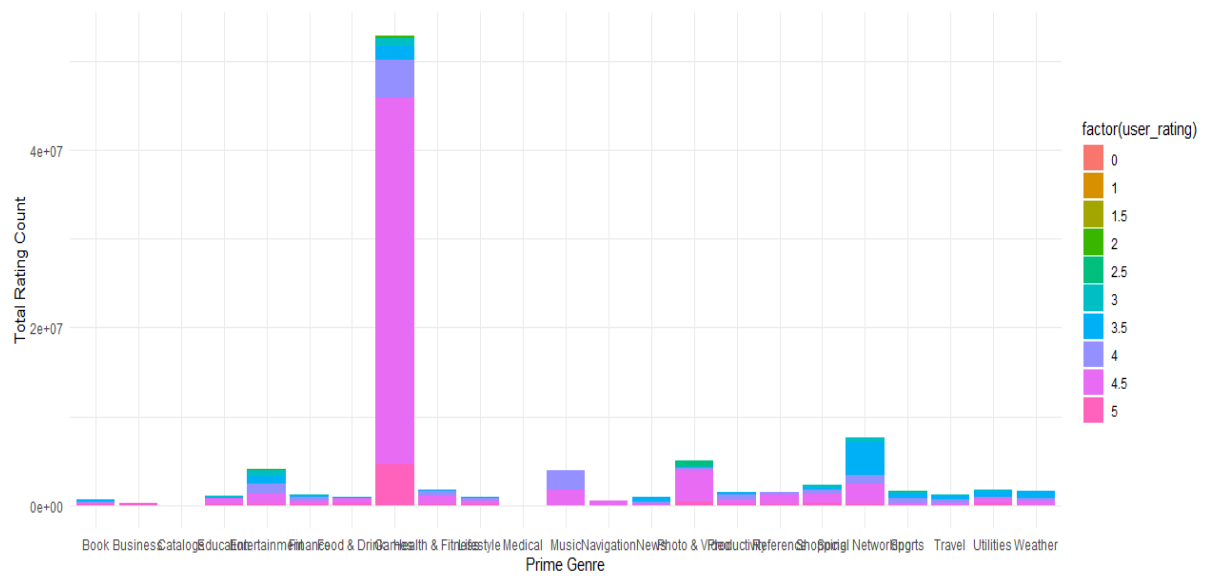


FIG 9 : Comparison between Prime genre and Total rating count

Result: Prime genre which is under games seems to have the maximum rating count

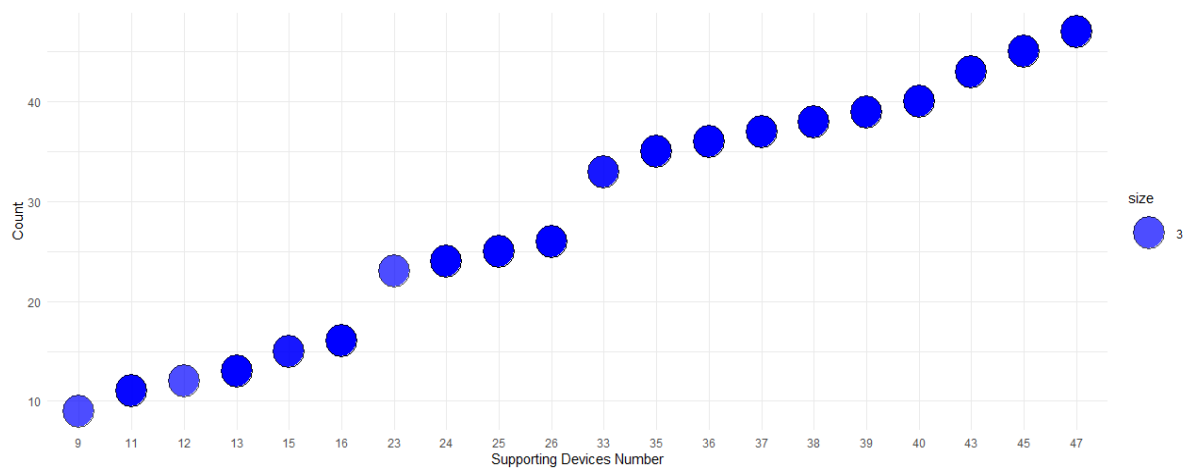


FIG 10 : Comparison between Supported device and Total rating count

Result : Apps that have support for 35-47 devices seems to have the maximum rating count

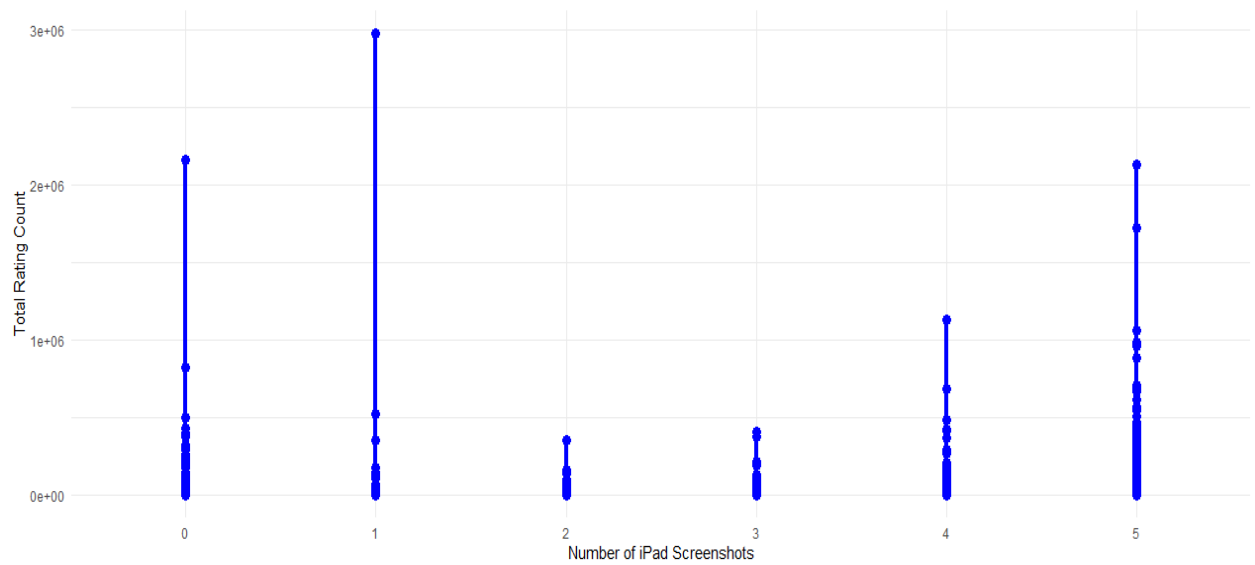


FIG 11 : Comparison between No of Screenshots and Total rating count

Result : Apps that have 1 screenshot displayed seems to have maximum rating count

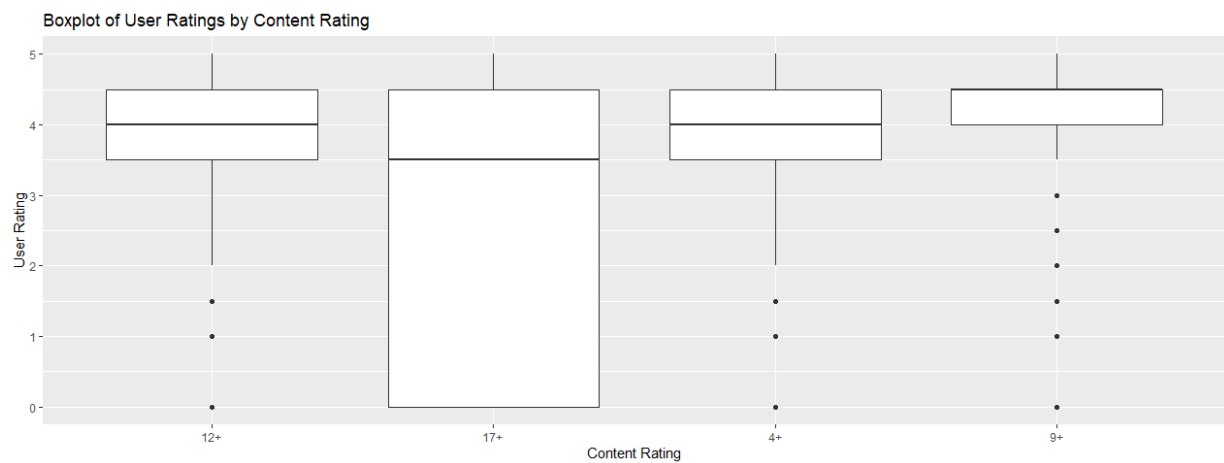


FIG 12 : Comparison between Content Rating(age) and User Rating count

Result : Apps that have 17+ age content rating seems to have maximum user rating count

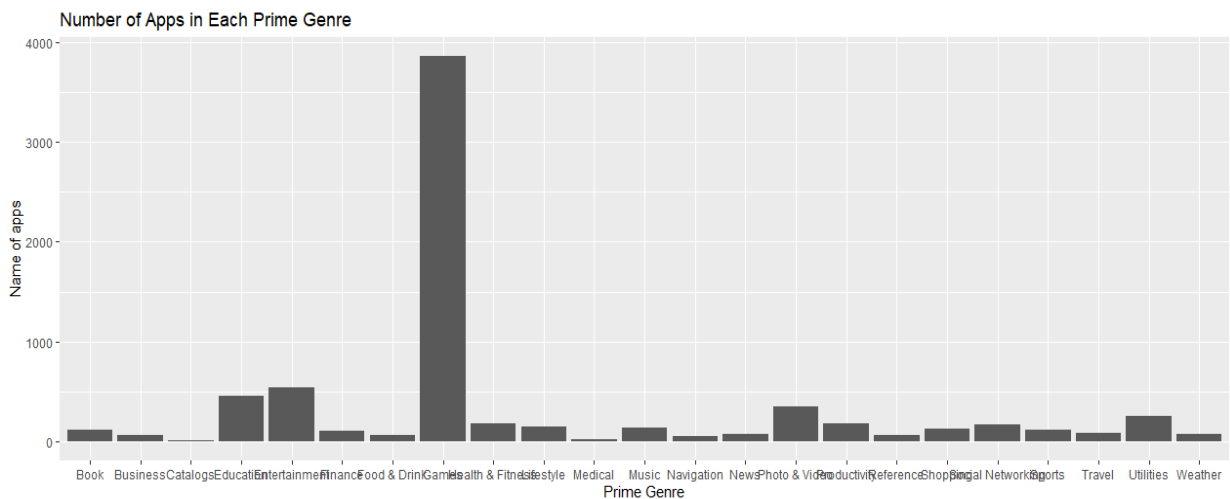


FIG 13 : No of Apps in each genre

Result : Game genre has highest number of apps

Based upon the analysis done above , high ratings have been recorded across apps that have the following characteristics:

Free app=**True**

No. of screenshots= **5**

No. of supported devices=**12**

Prime Genre=**social media**

No. of language support = **between 27-29**

app containing highest rating by age= **17+ age**

Based on paid status identifying the app count and most preferred genre:

11.) User rating by Prime genre and Paid Status=medical and shopping (**Lower paid**), music , weather (**Lower unpaid**)

12.) paid status count = **unpaid**

Machine Learning Models

Since the target attribute (successful_app) is a categorical field, classification models are excellent for predicting and evaluating the results. The classification models that are trained, tested, and assessed during the process are listed below. The machine learning models that will be used to carry out this task include those in supervised learning Logistic Regression, Support vector machine, Decision tree, Random Forest, KNN and Naïve bayes.

Binary classification

The purpose of binary classification is to categorise data into one of two classes or categories. Positive (1) and negative (0) are commonly used to express the two possible outcomes. Binary classifier performance is typically evaluated using metrics such as accuracy, precision, recall, and F1 score. In our dataset, we classified the data

based on the user rating, where a rating is < than 4 is considered 0 and a rating of >=4 is considered 1.

User Rating	Class
<4	0
≥4	1

Multiclass Classification

The objective of multiclass classification is to categorise occurrences into more than two classes. Each instance is assigned to a single class. Logistic regression (with extensions), decision trees, random forests, support vector machines, and deep learning models such as neural networks are examples of multiclass classification algorithms. Accuracy, precision, recall, F1 score, and confusion matrices are common evaluation measures for multiclass classification. We classified the data in our dataset based on the user rating, where a rating of 1 -2 is 1, 2-3 is 2, 3-4 is 3, and 4-5 is 4.

User Rating	Class
1 to 2	1
2 to 3	2
3 to 4	3
4 to 5	4

1.Logistic regression:

Logistic regression is ideal for binary classification situations in which we predict a single result (e.g., successful_app or not). It simulates the likelihood of an instance belonging to a specific class. The findings of logistic regression are clearly interpretable. The model's coefficients represent the strength and direction of the association between each predictor variable and the binary outcome's log-odds. By employing proper encoding strategies such as one-hot encoding, logistic regression may handle categorical predictor variables (e.g., cont_rating and prime_genre). As a result, it is appropriate for datasets containing a combination of numeric and categorical attributes. Logistic regression models produce probabilities ranging from 0 to 1. This can be useful if you need probabilistic forecasts, as it allows you to set a decision-making threshold based on the projected probabilities.

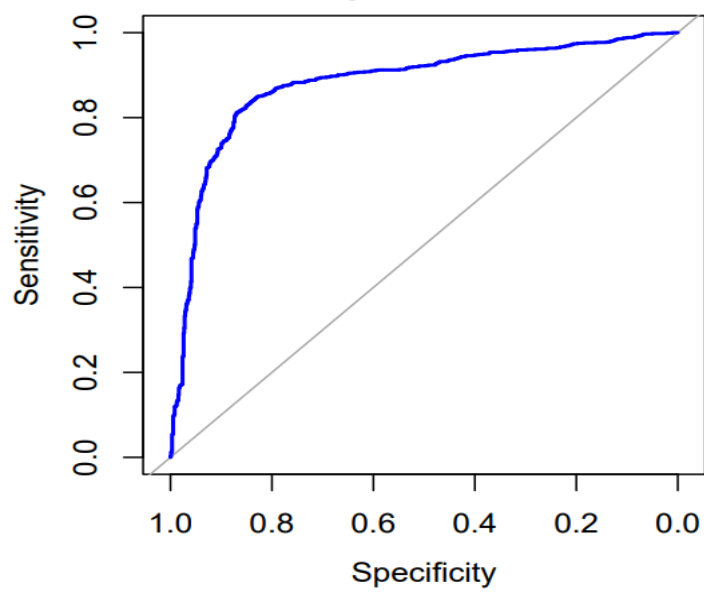
Binary classification

Model	Logistic Regression (Oversampled)
Evaluation score	Accuracy = 0.8111342
	Precision = 0.77090349
	Recall = 0.8830544
	F1 Score = 0.8230868

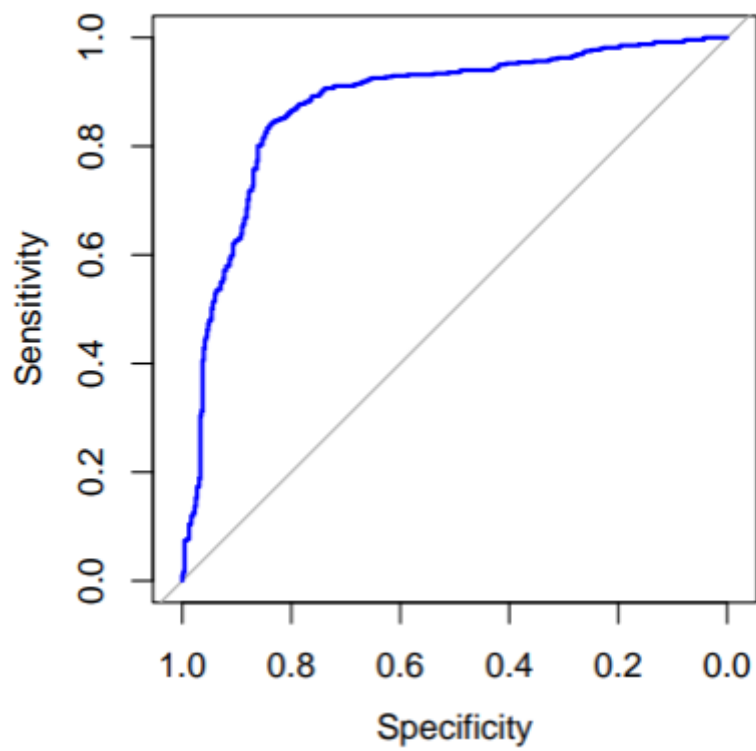
Model	Logistic Regression (Undersampled)
Evaluation score	Accuracy = 0.8186335
	Precision = 0.78200811
	Recall = 0.8853002
	F1 Score = 0.83018139

ROC CURVE

LR-OverSampled Data-ROC Curve



LR-UnderSampled Data-ROC Curve



2. Decision tree:

Decision trees are simple to interpret and comprehend. The dividing rules and the resulting tree structure provide explicit insights into the decision-making process, which makes it easier to explain to stakeholders. Decision trees can handle both numeric and categorical characteristics, which is useful for our dataset, which includes both types of variables. We have "numeric" and "character" variables, and decision trees can handle both without considerable preprocessing. The scale of numeric features has little effect on decision trees. As a result, performing feature scaling (e.g., normalization or standardization) before to using the algorithm is not required. Non-linear correlations between features and the goal variable can be captured via decision trees. This is especially beneficial when your data's associations are not linear, as decision trees can provide complex decision boundaries.

Binary classification

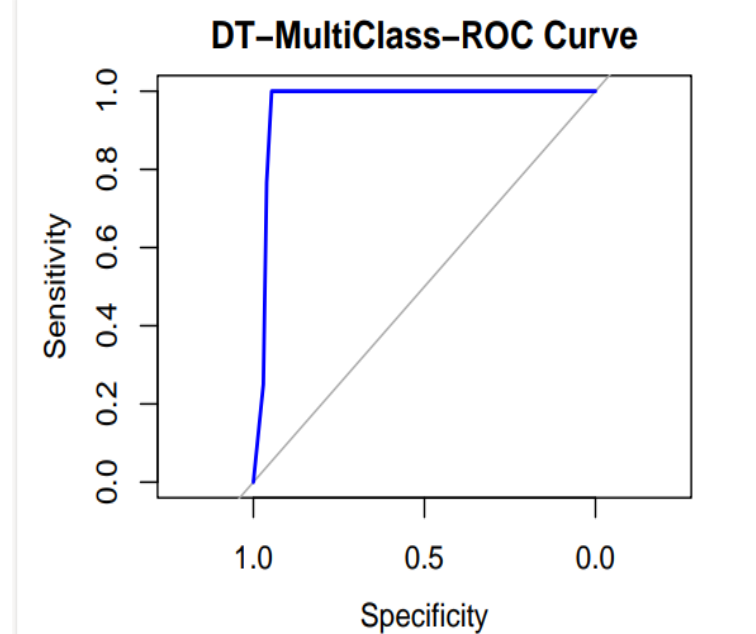
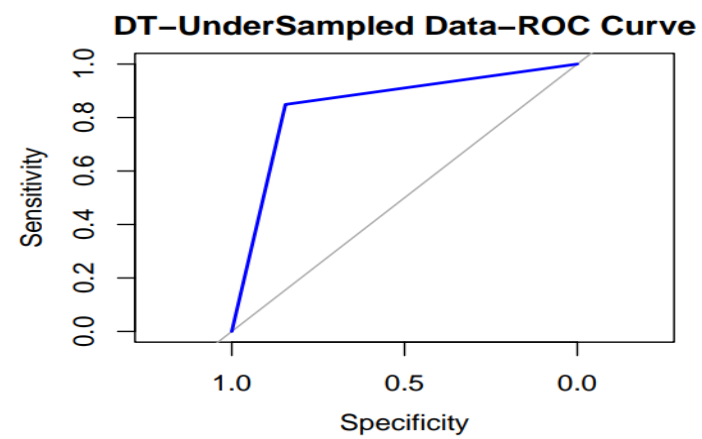
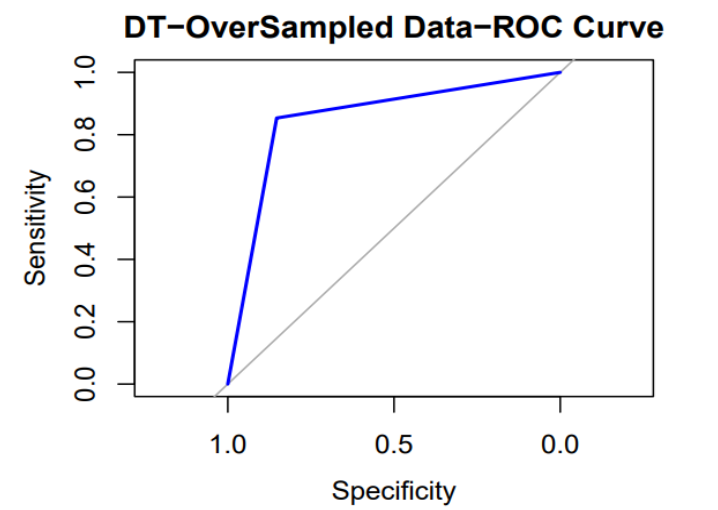
Model	Decision Tree Oversampled
Evaluation score	Accuracy = 0.8494277
	Precision = 0.8466527
	Recall = 0.84830119
	F1 Score = 0.8494133

Model	Decision Tree Undersampled
Evaluation score	Accuracy = 0.8515528
	Precision = 0.85248931
	Recall = 0.8509317
	F1 Score = 0.8515528

Multiclass Classification

Model	Decision Tree Multiclass
Evaluation score	Accuracy = 0.8396125
	Precision = 0.64739841
	Recall = 0.209071
	F1 Score = 0.28647699

ROC CURVE



3.Random Forest:

Random Forest can handle both numerical and categorical variables without requiring considerable pre-processing. The mix of "numeric" and "character" variables in our dataset is favourable in this regard. Random Forest is an ensemble learning method that trains by constructing several decision trees. By eliminating overfitting, this ensemble technique improves the model's generalisation performance and robustness. Random Forest assigns a feature relevance score to each variable, showing how much each feature contributes to the model's predictive performance. This can be useful for feature selection and determining which variables have the most influence. Random Forest can record variable interactions, allowing for a more comprehensive knowledge of how different features influence the target variable. When compared to individual decision trees, Random Forest frequently delivers excellent prediction accuracy and is less prone to overfitting. This is very useful when working with large datasets.

Binary classification

Model	Random Forest Oversampled
Evaluation score	Accuracy = 0.9212626
	Precision = 0.93063584
	Recall = 0.9076815
	F1 Score = 0.91901534

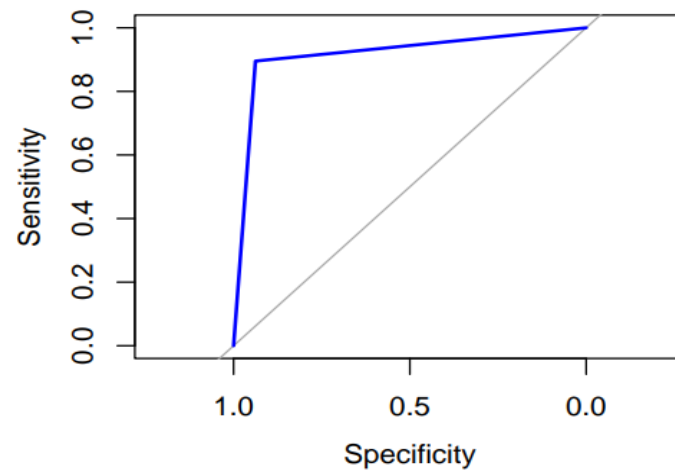
Model	Random Forest Undersampled
Evaluation score	Accuracy = 0.8502415
	Precision = 0.82959049
	Recall = 0.877095
	F1 Score = 0.8526816

Multiclass Classification

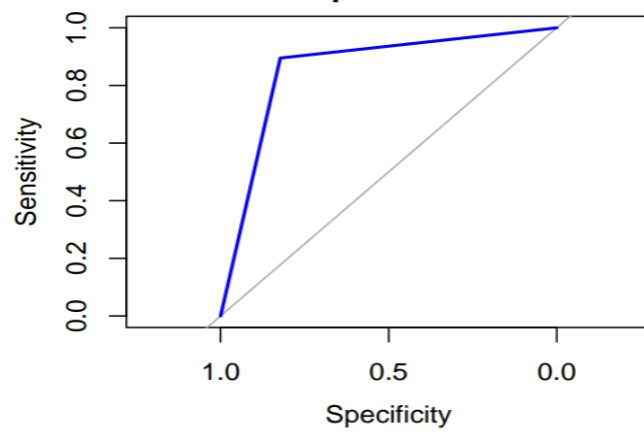
Model	Random Forest Multiclass
Evaluation score	Accuracy = 0.8429465
	Precision = 0.76923077
	Recall = 0.2985075
	F1 Score = 0.43010753

ROC CURVE

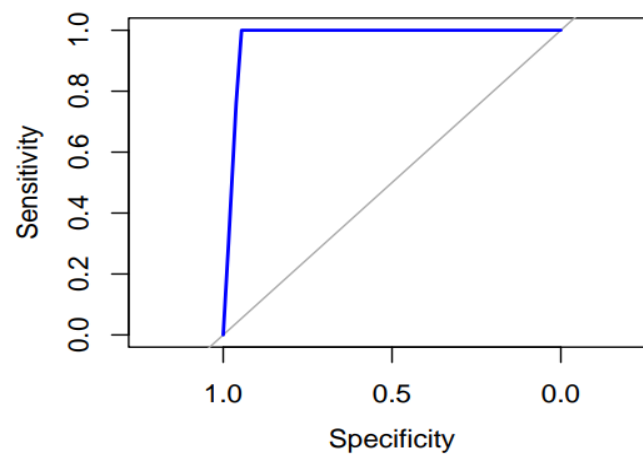
RF-OverSampled Data-ROC Curve



RF-UnderSampled Data-ROC Curve



RF-Multiclass-ROC Curve



4. Support vector machine:

SVM performs well in datasets with many dimensions (features). Because our dataset contains a variety of variables such as price, rating counts, user ratings, and others, SVM is an appropriate choice. SVM can effectively handle non-linear data relationships by utilising various kernel functions (e.g., polynomial, radial basis function). This is useful if your dataset's associations are not linear. SVM is less susceptible to overfitting, particularly in high-dimensional spaces. It generalises well to new, previously unseen data, making it appropriate for datasets with a reasonable number of features. SVM allows you to select alternative kernel functions based on the properties of your data. This adaptability enables customisation to capture various sorts of relationships. When features interact, SVM is effective. It can capture complex relationships by establishing non-linear decision limits. SVM is resistant to outliers in the dataset. Support vectors, rather than all data points, influence the decision boundary.

Binary classification

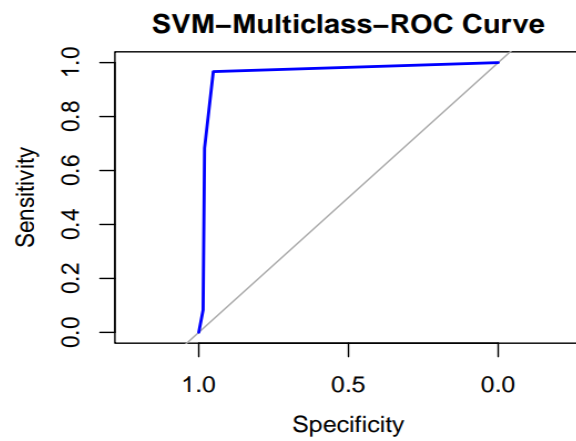
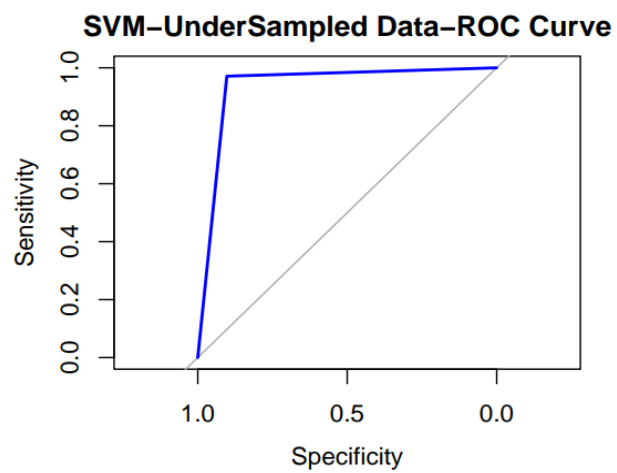
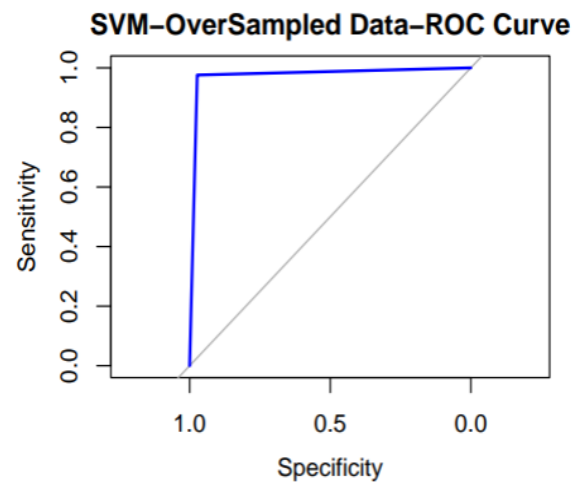
Model	Support vector machine Oversampled)
Evaluation score	Accuracy =0.9776275
	Precision = 0.97326258
	Recall = 0.9820084
	F1 Score = 0.97761038

Model	Support vector machine (Undersampled)
Evaluation score	Accuracy =0.947619
	Precision =0.93537948
	Recall =0.9619048
	F1 Score = 0.94841413

Multiclass Classification

Model	Support vector machine (Multiclass)
Evaluation score	Accuracy =0.9236978
	Precision =0.74401171
	Recall =0.2912022
	F1 Score =0.41576564

ROC CURVE:



5.KNN:

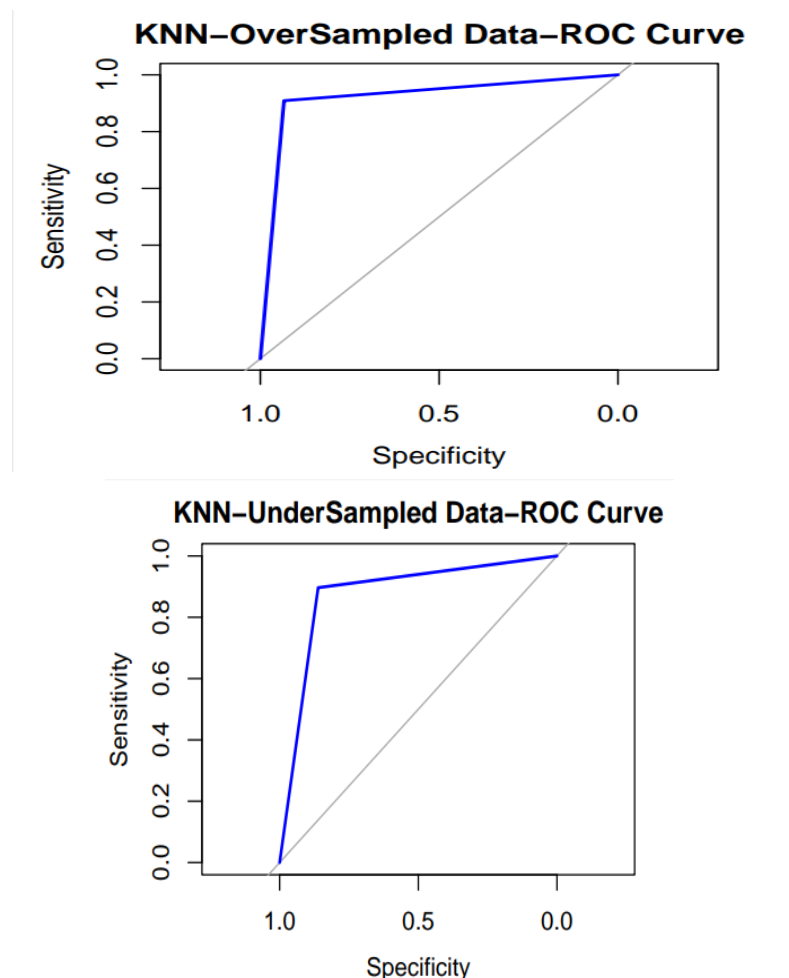
KNN is capable of handling both numerical and categorical features. KNN's ability to handle diverse data types without requiring considerable preparation is advantageous in your dataset, which has a mix of "numeric" and "character" variables. KNN is relatively resistant to dataset outliers. Because the predictions are based on the closest neighbours, a few outliers are unlikely to have a substantial impact on the outcomes. KNN is a non-parametric method, which means that no assumptions are made about the functional structure of the underlying relationships. This adaptability enables it to detect complicated patterns in data. KNN is especially good at detecting local patterns or structures in data. It works well when the decision boundaries are non-linear or the data contains clusters. KNN is an instance-based learning method that does not require any explicit training. During training, the model memorises the entire dataset, and predictions are generated based on the nearest neighbours at the time of prediction. This is useful in circumstances when the data is continually changing.

Binary classification

Model	KNN algorithm (Oversampled)
Evaluation score	Accuracy =0.9239334
	Precision =0.92969518
	Recall =0.9165272
	F1 Score = 0.92300806

Model	KNN algorithm (Undersampled)
Evaluation score	Accuracy =0.884472
	Precision =0.87776067
	Recall =0.8935818
	F1 Score = 0.88554386

ROC CURVE



6. Naive bayes:

Naive Bayes is a rapid and efficient computing method. It's especially useful for massive datasets and real-time applications that require speedy forecasts. Naive Bayes is a straightforward algorithm that is simple to grasp and apply. It frequently takes less training data than more complicated models, making it appropriate for scenarios with limited data. The Naive Bayes algorithm can handle both numerical and categorical features. The ability of Naive Bayes to accommodate diverse data types without considerable preparation is advantageous in your dataset, which has a mix of "numeric" and "character" variables. When dealing with categorical features, Naive Bayes works well. It works especially well in circumstances where the characteristics are discrete or represent categories. Because Naive Bayes has few hyperparameters to modify, substantial parameter tuning is not required. This can be useful, especially when determining the best parameters is difficult or time-consuming.

Binary classification

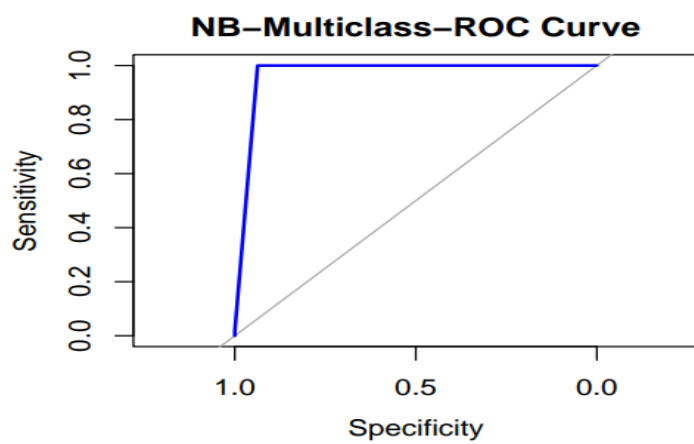
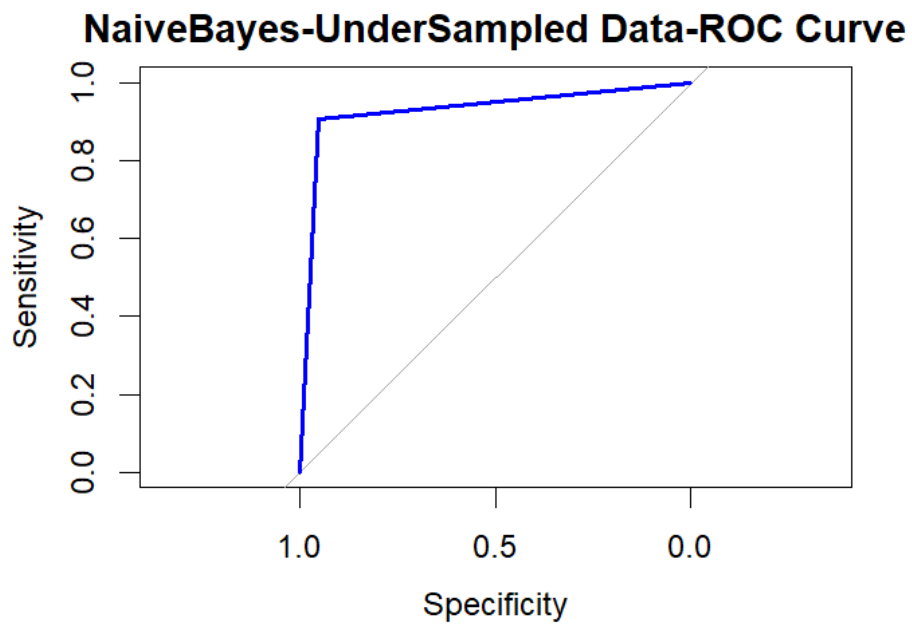
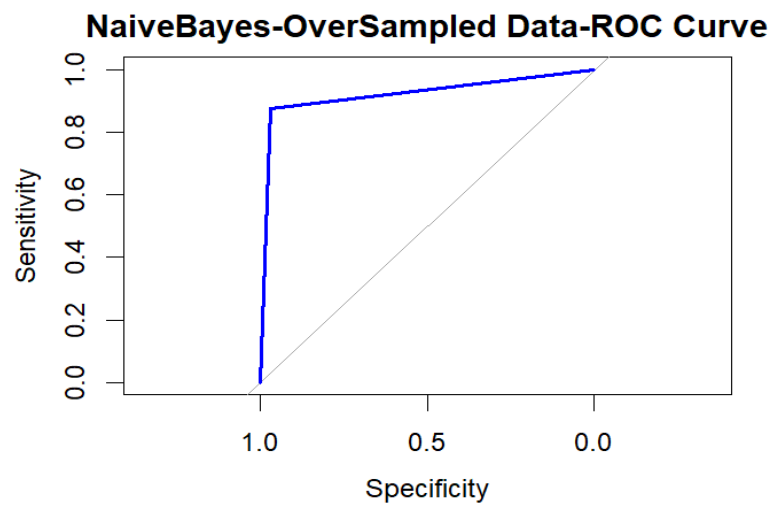
Model	Naïve bayes (Oversampled)
Evaluation score	Accuracy =0.921436
	Precision =0.96388877
	Recall =0.8751046
	F1 Score = 0.91654946

Model	Naïve bayes (Undersampled)
Evaluation score	Accuracy =0.9124224
	Precision =0.95934848
	Recall =0.8616977
	F1 Score = 0.9061583

Multiclass Classification

Model	Naïve bayes (Multiclass)
Evaluation score	Accuracy =0.7926355
	Precision =0.2324629
	Recall =0.9468852
	F1 Score = 0.37252779

ROC CURVE



Important features:

Focusing on the most important elements can improve model performance. By removing irrelevant or redundant characteristics, the model may become less prone to overfitting and more resistant to data perturbations. A model with fewer characteristics is typically simpler and easy to understand. This is critical for comprehending the elements influencing the forecasts, particularly in cases where interpretability is critical.

1.Random Forest:

A typical and effective strategy for feature selection is to build a new model using selected important features from the Random Forest model. The Random Forest model probably identified the features mentioned - older_ver_rating, rating_count_tot, rating_count_ver, and user_rating_ver - as essential.

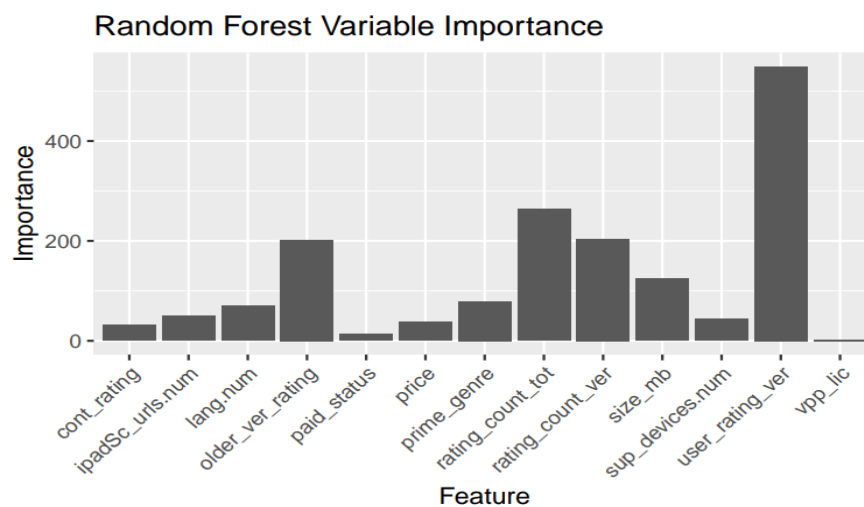


FIG 14: Random forest variable importance

2.Decision tree:

A valid strategy for feature selection is to build a new model utilising important features selected from a Decision Tree model. The Decision Tree model probably identifies the features you've mentioned as important: ipadSc_urls.num, older_ver_rating, rating_count_tot, rating_count_ver, and user_rating_ver.

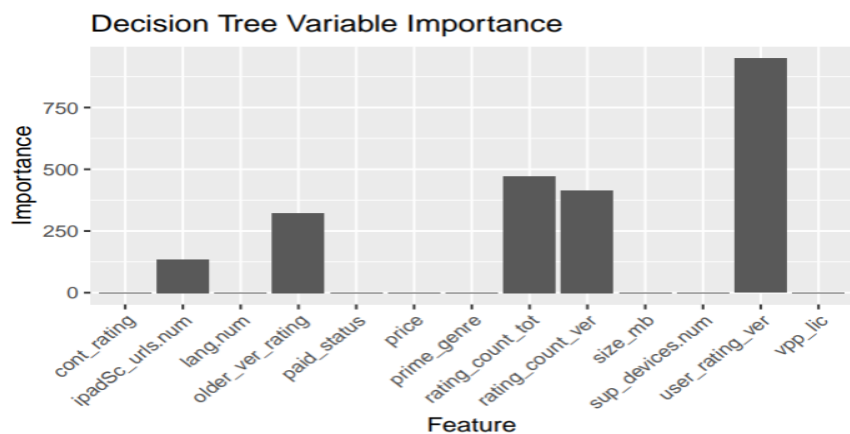


FIG 15: Decision tree variable importance

Evaluation method:

Evaluation metrics are used to analyse a machine learning model's performance. These metrics provide measurable indicators of how well the model performs. It is critical to choose measurements that are aligned with the desired objectives while considering into account the challenges of the problem.

Below is the comparison of metrics of all the models:

S.no	Model	Accuracy	Precision	Recall	F1 Score
1	LOGISTIC REGRESSION-KFold(over-sampled)	0.811134	0.770903	0.883054	0.823087
2	LOGISTIC REGRESSION-KFold(under-sampled)	0.818634	0.782008	0.8853	0.830181
3	RANDOM FOREST (oversampled)	0.921263	0.930636	0.907682	0.919015
4	RANDOM FOREST (undersampled)	0.850242	0.82959	0.877095	0.852682
5	RANDOM FOREST(Multiclass)	0.842947	0.769231	0.298508	0.430108
6	SVM-KFold(oversampled)	0.977628	0.973263	0.982008	0.97761
7	SVM-KFold(undersampled)	0.947619	0.935379	0.961905	0.948414
8	SVM-KFold(Multiclass)	0.923698	0.744012	0.291202	0.415766
9	NaiveBayes-KFold(oversampled)	0.921436	0.963889	0.875105	0.916549
10	NaiveBayes-KFold(undersampled)	0.912422	0.959348	0.861698	0.906158
11	NaiveBayes-KFold(Multiclass)	0.792636	0.232463	0.946885	0.372528
12	KNN-KFold(oversampled)	0.923933	0.929695	0.916527	0.923008
13	KNN-KFold(undersampled)	0.884472	0.877761	0.893582	0.885544
14	DECISION TREE-KFold(oversampled)	0.849428	0.850101	0.846653	0.848301
15	DECISION TREE-KFold(undersampled)	0.851553	0.852489	0.850932	0.85152
16	DECISION TREE-KFold(Multiclass)	0.839613	0.647398	0.209071	0.286477
17	RANDOM FOREST(oversampled-Important Features)	0.886923	0.919575	0.846369	0.881455
18	RANDOM FOREST(undersampled-Important Features)	0.828157	0.816994	0.851499	0.833889
19	DECISION TREE-KFold(oversampled-Important Features)	0.849428	0.850147	0.846653	0.848313
20	DECISION TREE-KFold(undersampled-Important Features)	0.849689	0.851413	0.847205	0.849176

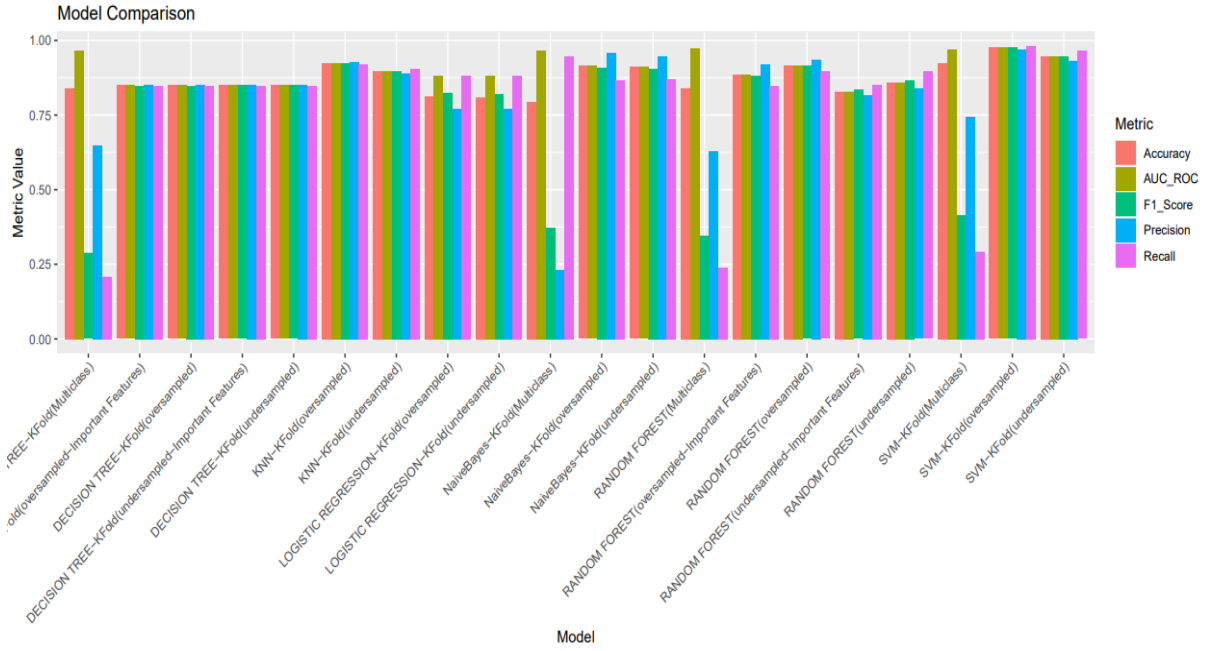


FIG 16: Comparison of all the models performed

Accuracy:

Accuracy is a fundamental evaluation parameter used in classification tasks to assess the overall correctness of a machine learning model's predictions. The formula for accuracy is as follows:

$$\text{Accuracy} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{Total Instances}}$$

Precision:

Precision is an evaluation parameter used in classification tasks to assess the accuracy of a machine learning model's positive predictions. It explicitly assesses the fraction of accurate positive predictions out of all positive predictions made by the model. The precision formula is as follows:

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

Recall:

Recall, also known as Sensitivity or True Positive Rate, is a classification task evaluation metric that assesses a machine learning model's ability to catch all relevant positive instances. It calculates the percentage of accurate positive predictions out of all actual positive events. The recall formula is as follows:

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

F1_Score:

In classification problems, the F1 Score is a measure that combines precision and recall into a single score. It strikes a balance between a model's ability to produce accurate positive predictions (precision) and record all positive instances (recall).

The F1 Score formula is as follows:

$$\text{F1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1 Score is a useful metric for balancing precision and recall, offering a single metric to evaluate the overall effectiveness of a classification model.

1.Binary classification:

Logistic Regression, Random Forest, SVM, Naive Bayes, KNN, and Decision Tree are among the binary models. Accuracy, Precision, Recall, and F1 Score are the evaluation measures examined.

Performance of the Model (Binary Classification with Imbalanced Data):

1.1 Logistic Regression Analysis:

Oversampling: An accuracy of 81.11% was achieved with a balanced precision-recall trade-off (precision: 77.09%, recall: 88.31%).

Under sampling yielded comparable results, showing consistency (accuracy: 81.86%).

1.2 Random Forest:

Oversampling: Excellent results with high accuracy (92.13%), precision (93.06%), and recall (90.77%).

Undersampling: Overall good performance, with balanced precision (82.96%) and recall (87.71%).

1.3 SVM:

Oversampling: Excellent performance with good accuracy (97.76%), balanced precision (97.33%), and recall (98.20%).

Undersampling: Excellent results with high precision (93.54%) and recall (96.19%).

1.4 Naive Bayes:

Oversampling: High accuracy (92.14%), precision (96.39%), and recall (87.51%) were achieved.

Undersampling performed well overall, with good precision (95.93%) and balanced recall (86.17%).

1.5 KNN:

Oversampling: High accuracy (92.39%), balanced precision (92.97%), and recall (91.65%).

Undersampling: Overall good performance, with balanced precision (87.78%) and recall (89.36%)

1.6 Decision Tree:

Oversampling: Yielded acceptable accuracy (84.94%), with balanced precision (85.01%) and recall (84.67%).

Undersampling Yielded comparable results, demonstrating consistency (accuracy: 85.16%).

2.Multiclass classification:

The performance measures of multiple machine learning models in a multiclass classification situation with imbalanced data. Random Forest, SVM, Naive Bayes, and Decision Tree are among the models. Accuracy, Precision, Recall, F1 Score, and the Area Under the Receiver Operating Characteristic (ROC) Curve are the evaluation criteria examined.

2.1 Random forest:

Overall accuracy of 84.29% was achieved.

High precision (76.92%) but low recall (29.85%), indicating probable difficulties in identifying specific classes.

The F1 score and ROC curve performance are both acceptable.

2.2 SVM:

Achieved high accuracy (92.37%) by balancing precision (74.40%) and recall (29.12%).

A high ROC curve score indicates that the model performs well in terms of discrimination.

2.3 Naive bayes:

The accuracy was moderate (79.26%), with a high recall (94.69%) but a low precision (23.25%).

The F1 score is acceptable, and the ROC curve indicates good discriminative power.

2.4 The Decision Tree:

Accuracy (83.96%) was good, with intermediate precision (64.74%) and low recall (20.91%).

The F1 score and ROC curve performance are also low.

Summary of binary and multiclass classification

Binary:

The top-performing model is SVM (Oversampled), with the highest accuracy and a balanced precision-recall trade-off. This shows that it is effective in coping with skewed datasets.

Random Forest (Oversampled) also performs well, with good accuracy and balanced precision-recall metrics.

Naive Bayes (Oversampled) and KNN (Oversampled) produce competitive outcomes, reaching high accuracy while balancing precision and recall.

Logistic Regression, Decision Tree, and SVM (Undersampled) all perform consistently, demonstrating that under sampling can be useful in some situations

Multiclass:

SVM stands out for its excellent accuracy, but at the expense of precision and recall. The ROC curve implies that the discriminative capacity is strong.

Random Forest has decent overall accuracy, but it requires monitoring owing to uneven recall across classes.

The Naive Bayes algorithm has a good recall but a low precision. Precision is essential in applications that require it.

Decision Tree has good accuracy however it has an uneven precision and recall. Additional tuning or exploration of ensemble approaches may improve its performance.

Important features:

We have implemented Decision tree and Random Forest model by identifying important variables from the binary classification model built before. We didn't achieve any improvement in the performance. This might be due to the collinearity between the variables.

Conclusion:

From the above interpretation we can identify that "SVM with oversampling for binary classification" achieves the best overall performance, whereas Random Forest with oversampling also performs effectively. Logistic Regression and Naive Bayes offer significant improvements, especially when oversampled. Decision Tree and KNN also produce competitive results, with oversampling improving performance. Furthermore, using oversampling enhances model performance across different techniques, emphasizing the necessity of dealing with imbalanced datasets.

So, the business can use the models built to understand the importance of the features/description of the app that are playing an effective role in the success of the app. Also, they can identify which features are lagging and does not contribute to the performance of the app and concentrate on it to improve the app performance.

Appendix

References:

Jordan, J. (2017). Hyperparameter tuning for machine learning models. [online] Jeremy Jordan. Available at: <https://www.jeremyjordan.me/hyperparameter-tuning/>.

Brownlee, J. (2020). Train-Test Split for Evaluating Machine Learning Algorithms. [online] Machine Learning Mastery. Available at: <https://machinelearningmastery.com/train-test-split-for-evaluating-machine-learning-algorithms/>.

Google (2019). Classification: ROC Curve and AUC | Machine Learning Crash Course. [online] Google Developers. Available at: <https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc>.

IBM Cloud Education (2020). What is Random Forest? [online] www.ibm.com. Available at: <https://www.ibm.com/cloud/learn/random-forest>.

knowledgeburrow.com. (n.d.). What is CRISP-DM model? – KnowledgeBurrow.com. [online] Available at: <https://knowledgeburrow.com/what-is-crisp-dm-model>.

IBM Cloud Education (n.d.). What is a Decision Tree | IBM. [online] www.ibm.com. Available at: <https://www.ibm.com/uk-en/topics/decision-trees>.

Kwak, S.K. and Kim, J.H. (2017). Statistical data preparation: management of missing values and outliers. Korean Journal of Anesthesiology, 70(4), p.407. doi:10.4097/kjae.2017.70.4.407.

Rodrigues, I. (2020). CRISP-DM methodology leader in data mining and big data. [online] Medium. Available at: <https://towardsdatascience.com/crisp-dm-methodology-leader-in-data-mining-and-big-data-467efd3d3781>.

"Introduction to Machine Learning with Python: A Guide for Data Scientists" by Andreas C. Müller and Sarah Guido

"A Few Useful Things to Know About Machine Learning" by Pedro Domingos

www.rdocumentation.org. (n.d.). scale function | R Documentation. [online] Available at: <https://www.rdocumentation.org/packages/base/versions/3.6.2/topics/scale>.

Ray, S. (2019). A Quick Review of Machine Learning Algorithms. [online] IEEE Xplore. doi:10.1109/COMITCon.2019.8862451.

IBM Cloud Education (2021). What is Overfitting? [online] www.ibm.com. Available at: <https://www.ibm.com/cloud/learn/overfitting>

Author Contribution:

S.no	Milestone description	Assigned	Signature
1	Basic planning		
1.1	Review and selecting usable datasets	All	
1.2	Determine the steps required to complete the project plan.	KGP	KGP
1.3	Discuss and plan the framework of the report.	JS	JS
2	Exploration and Preprocessing of Data		
2.1	Explore the dataset for trends.	AVS	AVS
2.2	Determine appropriate metrics for analysis based on preliminary model designs.	MA	MA
2.3	In the report, describe the exploration, pre-processing steps, and findings.	JAAJS	JAAJS
2.4	Prepare and partition the dataset.	JS	JS
3	Modelling		
3.1	Basic model building for binary classification	All	
3.1.1	Logistic Regression	AVS	AVS
3.1.2	Decision Tree	JAAJS	JAAJS
3.1.3	Random Forest	JAAJS	JAAJS
3.1.4	SVM	KGP	KGP
3.1.5	KNN	JS	JS
3.1.6	Naïve Bayes	MA	MA
3.2	Model building for multiclass classification	AVS	AVS
3.2.1	Decision Tree	JAAJS	JAAJS
3.2.2	Random Forest	AVS	AVS
3.2.3	Naïve Bayes	MA	MA
3.2.4	SVM	KGP	KGP
3.3	Identification of important features of each model and build a model out of it	JS	JS
4	Evaluation		
4.1	Collect data to evaluate model performance.	KGP, AVS	KGP, AVS
4.2	Plan, investigate, and analyse	JAAJS	JAAJS
4.3	In the report, describe and assess the findings.	JS, MA	JS, MA
4.4	Working on final report	All	

Script details:

Script details and order of script execution:

Scripts Explanation:

Data Cleaning.R: Read the data, checked for data discrepancies.

Feature Engineering.R: Created new features and transformed existing features as per the project requirements.

Data Preparation for Modelling-Binary Classification.R: Created dataframes for the modelling by creating binary target variable and performed Scaling and Encoding. To handle data imbalance did Random sampling (UnderSampling and overSampling).

Data Preparation for Modelling-MultiClass Classification.R: Created dataframes for the modelling by creating performed Scaling and Encoding. #Logistic Regression-KFold.R : Performed Binary Classification both OverSampled and UnderSampled Data KFold.

Decision Tree-KFold.R: Performed Binary Classification both OverSampled and UnderSampled Data using KFold.

Random Forest.R: Performed Binary Classification both OverSampled and UnderSampled Data using KFold.

KNN-KFold.R: Performed Binary Classification both OverSampled and UnderSampled Data using KFold.

Naive Bayes-KFold.R: Performed Binary Classification both OverSampled and UnderSampled Data using KFold.

Logistic regression-KFold.R: Performed Binary Classification both OverSampled and UnderSampled Data using KFold.

Binary Classification-Full.R: Single script which contains the code for all the binary classification models.

Multiclass Classification-Full.R: Single script which contains the code for all the MultiClass classification models.

Evaluation Table.R: Script to create a table to append and compare the metrics of each model.

Important Features-Graph.R: Script to find the important features that are identified in each model.

Decision Tree-Important Features.R: Performed Decision Tree by identifying the important features from binary classification model.

Random Forest-Important Features.R: Performed Random Forest by identifying the important features from binary classification model.

PBA_EDA.R: Performed data exploration of all the features using plots.

Order of Execution of the scripts: For Binary Classification:

Step-1: Execute Data Cleaning.R:

Step-2: Execute Feature Engineering.R

Step-3: Execute Data Preparation for Modelling-Binary Classification.R

Step-4: Execute Evaluation Table.R

Step-5: Execute Logistic Regression-KFold.R, Decision Tree-KFold.R, Random Forest.R, KNN-KFold.R, Naive Bayes-KFold.R, Naive Bayes-KFold.R in sequence or can also execute the Binary Classification-Full.R.

For MultiClass Classification:

Step-1: Execute Data Cleaning.R

Step-2: Execute Feature Engineering.R

Step-3: Execute Data Preparation for Modelling-MultiClass Classification.R

Step-4: Execute Evaluation Table.R

Step-5: Execute Multiclass Classification-Full.R